Ders 9 Genişletilmiş Notu: Mediator Kalıbı, Refactoring'e Giriş, JUnit ile Birim Sınaması ve İlk Kod Kusurları

Ders 9 Genişletilmiş Notu: Mediator Kalıbı, Refactoring'e Giriş, JUnit ile Birim Sınaması ve İlk Kod Kusurları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Mediator Kalıbı: Amaç ve Font Seçme Penceresi Örneği
- 2. Mediator Kalıbının Yapısı, Zayıf Yönleri ve Kullanım Yeri
- 3. Refactoring Nedir ve Neden Yapılır?
- 4. Kod Kusurları (Code Smells)
- 5. Birim Sınaması ve Yerine Geçen Kod
- 6. JUnit 3.x ile Test Case Hazırlamak
- 7. JUnit 4.x Gelişmeleri ve Annotation Kavramı
- 8. Video Kiralama Örneği
- 9. Video Kiralama Örneğinde JUnit Uygulaması
- 10. Duplicated Code ve Extract Method
- 11. Long Method ve İkinci Extract Method Örneği
- 12. Yinelenen Kodun Yerini Bulmak: Pull Up Method ve Form Template Method
- 13. Decompose Conditional
- 14. Long Parameter List: Introduce Parameter Object ve Preserve Whole Object
- 15. Replace Temp with Query ve Başarım Tartışması
- 16. Replace Method with Method Object
- Örnek Sorular
- Dersin Toparlaması

1. Mediator Kalıbı: Amaç ve Font Seçme Penceresi Örneği

Bir önceki mevcut kayıtta (ders 7) davranışsal kalıplardan Strategy, State, Template Method ve Visitor işlenmişti. Bu ders, tasarım kalıpları konusunu bitiren Mediator kalıbıyla başladı. Mediator Türkçeye "ara bulucu" diye çevrilebilir.

1.1 Amaç

Kalıbın amacı, bir nesne grubunun arasındaki etkileşimleri sarmalamaktır. Bunun sonucu olarak nesneler birbirleriyle doğrudan değil, dolaylı olarak ilişki kurar. Kalıbın getireceği avantaj şudur: bu nesneler arasındaki ilişkilerde bir değişiklik yapmak gerektiğinde, bu değişikliği nesnelerin kodunu değiştirmeden yapabiliriz.

1.2 Font seçme penceresi

Hoca kalıbı bir örnekle açıkladı (ekrandan, 00:10:00). Bir font seçme penceresi (Font Chooser) hazırlanacak. Pencerede şu bileşenler var:

- Fontun ailesini seçen bir giriş alanı ve altında aile listesi (avant garde, chicago, courier, helvetica, new century schoolbook, palatino, times roman, zapf dingbats).
- Weight satırında medium, bold ve demibold radyo düğmeleri.
- Slant satırında roman, italic ve oblique radyo düğmeleri.
- Size alanı (örnekte 34 pt) ve condensed onay kutusu.
- Cancel ve OK düğmeleri.

Bileşenlerde yapılan her değişiklik, en üstteki önizleme alanına ("The quick brown fox…") anında yansımalıdır. Italic seçiliyken yazı italiktir; roman seçilince italiklik hemen kalkar. Bold kaldırılıp normal ağırlığa geçilince yazı normal ağırlıkla görünür. Kodlanmak istenen, bu anlık etkileşimlerdir.

Bileşenler yalnız önizlemeyi etkilemez, birbirlerini de etkiler. Medium ve bold aynı radyo grubundadır: biri seçilince diğerinin seçimi kalkar. Roman ile italic de aynı biçimde birbirine bağlıdır. Yani bu parçalar hem birbirleriyle hem de önizleme alanıyla yakın etkileşim içindedir.

Hoca burada akla Observer gelebileceğini, ama Observer kullanmanın bir anlamı olmadığını söyledi; burada yapılacak işler daha anlık ve karşılıklıdır.

1.3 Bağlantı sayısı

Hoca, en kötü senaryoda her bileşenin her bileşenle haberleştiğini düşündü. Dört bileşen birbirleriyle çift yönlü haberleşirse bağlantı sayısı:

$$4 \cdot 3 = 12$$

Beşinci bir bileşen eklendiğinde, önceki eleman sayısı kadar bağlantı gelir; ama iki yönlü olduğu için bunun iki katı kadar:

$$4 \cdot 2 = 8 \quad \Rightarrow \quad 12 + 8 = 20 = 5 \cdot 4$$

Genel olarak n bileşende $n(n-1)$ yönlü bağlantı vardır (genel formül ara adım olarak eklendi). Bileşen ekledikçe bağlantı sayısı karesel büyür.

Çözüm, etkileşimi merkezi bir noktada ele almaktır: bileşenler birbirleriyle değil, merkezi bileşenle haberleşir; merkezi bileşen iletişimleri sarmalar ve gerekli yönlendirmeleri yapar. Bu tasarım Mediator kalıbını verir. Ekrandaki nesne şemasında `aClient`, `aListBox`, `aButton` ve `anEntryField` nesnelerinin her birinde bir `director` alanı vardır ve hepsi `aFontDialogDirector` nesnesine bağlıdır; `aFontDialogDirector` de bileşenlere bağlıdır (ekrandan, 00:10:00).

2. Mediator Kalıbının Yapısı, Zayıf Yönleri ve Kullanım Yeri

2.1 Hazır GUI çatısı üzerinde

GUI çatılarında bileşenler zaten `Widget` soyut sınıfının alt sınıflarıdır: liste kutusu, giriş alanı, radyo düğmesi hep `Widget`'tan türer. Diyalog kutusu oluşturmak için soyut bir sınıf ve soyut metotlar, bir GUI bileşeni değiştiğinde çağrılacak metot da çatıda hazırdır. Biz bundan kalıtımla kendi `FontDialogDirector` sınıfımızı oluştururuz. Bu yüzden hocanın gösterdiği sınıf şemasının büyük kısmı kalıp hakkında bir şey söylemez; yalnız hocanın kırmızıyla işaretlediği kısım kalıbın uygulandığı yerdir.

2.2 Genel yapı

Birbiriyle haberleşecek bileşenler elimizde hazır değilse onları da biz oluştururuz. O zaman kalıbın genel sınıf şeması şöyledir (ekrandan, 00:14:30):

```
Mediator (soyut)  <---- mediator ----  Colleague (soyut)
     ^                                      ^
     |                                      |
ConcreteMediator ---> ConcreteColleague1, ConcreteColleague2
```

Colleague “meslektaş” demektir; daha modern bir terimle bunlara peer de denir. Colleague’lerin ana davranışını soyut bir sınıf belirler, somut colleague sınıfları ondan türer. Normalde birbirleriyle konuşacak olan colleague’ler kalıp uygulandığı için artık bir Mediator üzerinden konuşur. Mediator’ın genel üyeleri ve metotları soyut `Mediator` sınıfında toplanır; somut `ConcreteMediator` sınıfları ondan türetilir.

- `ConcreteMediator`, somut colleague nesnelerinin bir koleksiyonunu tutar. Onlara ulaşarak birbirleriyle görüşmelerini yürütür: durumlarını alır, değiştirir, ne gerekiyorsa yapar.
- Colleague’ler, soyut üst sınıflarındaki `mediator` üye alanı sayesinde `ConcreteMediator`’ı dolaylı olarak görür.
- Nesneler arasındaki bütün etkileşimler `ConcreteMediator` nesnesi üzerinden yapılır.
- Haberleşmenin çift yönlü olması şart değildir; kalıp tek yönlü ve çift yönlü haberleşmeyi destekler.

2.3 Zayıf yönü

Hocaya göre bu kalıp, bütün dağınıklığı halının altına ya da dolabın içine tıkıştırmak gibi bir çözümdür. Mediator gerçeklemeleri çok özelleşmiştir; bu yüzden yeniden kullanımları zordur. Her işbirliği senaryosu için kalıbı sıfırdan kurmak ve Mediator’ı sıfırdan kodlamak gerekir.

“Nesne sınıflarının kodunu değiştirmeye gerek kalmaz” diye kalıbın reklamı yapılır, ama hiçbir şey kendiliğinden, sihirli bir şekilde olmaz: değişiklikler bu kez karmaşık Mediator sınıfının içinde yapılır.

2.4 Ne zaman yararlıdır?

Bir grup nesne arasında iyi tanımlanmış ama karmaşık etkileşimler varsa kullanılır. “İyi tanımlanmış” olması, mümkün olduğunca az değişeceği anlamına gelir; çünkü değiştirilecek Mediator sınıfı gerçekten karmaşıktır.

Kalıbın getirebileceği avantajlar:

- Bileşenlerin kaynak kodu bizde yoksa zaten onlara dokunamayız; Mediator tek yoldur.
- Bir uygulama güncellemesi dağıtırken, kullanıcılara önceden gönderilmiş sınıfların kodu değişmez; yalnız Mediator’daki güncellemeyi dağıtmak yeter.

Davranış daha fazla değişirse iş zorlaşır. Hoca kalıp çok ayrıntılı olmadığı için uygulama örneği yapmadı. Birazdan görülecek kod kusurları açısından Mediator’ın bir “large class” olduğunu, sonradan yapılacak küçük düzenlemelerle karmaşıklığının bir noktaya kadar azaltılabileceğini söyledi. Böylece tasarım kalıpları konusu bitti.

3. Refactoring Nedir ve Neden Yapılır?

3.1 Hatalardan öğrenmek

Tasarım kalıplarıyla nesneye dayalı tasarımın doğru nasıl yapılacağı öğrenildi. Öğrenmenin önemli bir bileşeni de hatalardan öğrenmektir. Dersin bu kısmında nesneye dayalı yaklaşım kullanılırken sık yapılan yanlış uygulamalar ele alınır. Hoca bunlara “bug” değil, “uygun olmayan uygulamalar” dedi. Amaç bunları yapmamak ve fark edildiklerinde nesneye dayalı yaklaşımın istediği biçime getirmeyi öğrenmektir.

3.2 Tanım

Slayttaki tanımlar (ekrandan, 00:20:00):

- Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmeye refactoring adı verilir.
- Yazılımın dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesidir.

Hoca ikinci tanımda “dışarıdan görünen davranış” ifadesini biraz açtı: yazılımın dışarıya açtığı kullanım biçimi, hatta API’si değişmeden iç yapısı değişir. Burada kastedilen Java’daki `interface` yapısı değil, dışarıya sunulan kullanım şeklidir. Yeni bir gereksinim eklemek ya da bir gereksinim değişikliğini kodlamak refactoring değildir. Zaten doğru çalışan kodda, örneğin

sınama ya da bir gereksinim değişikliği sırasında yapılması gereken işlerin olması gerekenden fazla olduğu hissedildiğinde yapılır. Hoca bunu sonrası için bir yatırım olarak düşünmeyi önerdi.

3.3 Amaç

Amaç, kodun iç kalite özelliklerini daha iyiye götürecek değişiklikler yapmaktır (ekrandan, 00:20:00):

- Kodun anlaşılabilirliğinin artırılması
- Koda esneklik kazandırılması
- Yeniden kullanılabilirliğin artırılması
- Yüksek uyum ve düşük bağımlılığın sağlanması
- Kalite denince akla gelen başka şeyler

Bir üye alanın ya da metodun adını değiştirip daha anlaşılır bir ad vermek bile refactoring'dir, çünkü anlaşılabilirliği artırır.

3.4 Nasıl ve ne zaman?

En doğrusu çevik yaklaşımdır: küçük adımlarla, küçük artımlarla ve sürekli sınama eşliğinde. Elimizde mevcut gereksinimleri karşılayan bir ilk tasarım vardır; bu tasarımın mükemmel olması ve her şeyi öngörmesi gerekmez. Çevik süreçler zaten "yeterince iyi" bir tasarımla gerçeklemeye başlamayı söyler. Önceki tasarım adımlarından birinde yanlış karar verildiği fark edilince refactoring yapılır. Değişikliklerin yeni hata doğurmadığından emin olmak için kod yeni haliyle sınanmalıdır; bu yüzden derste birim sınamasına da girildi.

Refactoring'e kodun toparlanmaya ihtiyacı olduğu hissedildiğinde başlanır. Bu şöyle anlaşılır: koda yeni özellik eklemek zorlaşır, bir hatayı bulmak zorlaşır, bulunan hatayı düzeltmek daha da zorlaşır.

4. Kod Kusurları (Code Smells)

İngilizcede "it smells" denince genelde kötü bir koku kastedilir. Kod açısından code smell:

- Yazılan kodun iyileştirilmesi gerektiğinin işaretleridir.
- Başka bir tanımla, kod içinde karşılaşılan hastalıklardır.
- Hocanın en önemli bulduğu tanımla, kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir. "Suboptimal" ya da "uygun olmayan" da denebilir; hoca Türkçede "kusurlu" demeyi tercih etti. Kod yapması gereken işi yapıyor mu? Yapıyor. Doğru mu yapıyor? Yapıyor, ama kusurlu yapıyor.

Yazılımın zamanla yamalı bohçaya dönmesi kaçınılmaz bir eğilimdir. Farklı zamanlarda farklı kişiler kodun farklı kısımlarında, ihtiyaç oldukça ve yalnız o kısmı düşünerek değişiklik yapar. Hepsi önceden öngörülebilseydi optimal bir tasarım yapılabilirdi; ama bu mümkün değildir. Bakım masrafları tavana vurmadan bu tehlike işaretlerine bakmak gerekir; bunun için refactoring yapılır.

Refactoring eylemleri, kodda karşılaşılan kusurları düzeltmek üzere yapılan yeniden yapılandırma eylemleridir. Yapılan bir refactoring yeni kod kusurları ortaya çıkarabilir; onları düzeltmek için yeniden refactoring yapılır.

Hoca, ilk haftalarda söylediği üzere Martin Fowler'ın "Refactoring" kitabını kullandığını ve oradaki refactoring'lerden seçilmiş bir kısmı ders notlarına aktardığını söyledi. Hepsini incelemeye dersin süresi yetmez. Kod kusurları ile refactoring eylemleri arasındaki ilişki bire bir değildir, çoka çoktur: bir kod kusuru birden fazla refactoring eylemiyle düzeltilebilir, bir refactoring eylemi de birden fazla kod kusurunu düzeltebilir. Hoca bu ilişkilerin en yoğun olduğu öbeği seçerek notlara eklemiş.

> **Ek bilgi (derste anlatılmadı):** Martin Fowler'ın "Refactoring: Improving the Design of Existing Code" kitabının ilk baskısı büyük olasılıkla 1999'da çıkmıştır. Kod kusurları bölümünü Kent Beck ile birlikte yazmıştır; video kiralama örneği kitabın ilk bölümündedir.

5. Birim Sınaması ve Yerine Geçen Kod

5.1 Tanım ve ne zaman tasarlanacağı

Hoca sınamanın esasının yazılım sınama üzerine verilen ayrı bir derste anlatıldığını, kendisinin burada koda daha yakın bir anlatım yapacağını söyledi.

Birim sınaması en küçük yazılım bileşeninin sınanmasıdır; nesneye dayalı programlamada bu, bireysel sınıfların sınanmasıdır (ekrandan, 00:30:00). Sınamalar üç ayrı zamanda tasarlanabilir: kodlamadan önce, kodlama sırasında ya da kodlamanın ardından.

- Kodlamadan önce tasarlanırsa buna Test Driven Development (test güdümlü geliştirme) denir. TDD yanlılarının sloganı: tek satır iç mantık kodu yazmadan önce sınamalarınızı hazırlayın. Gerekçeleri, yeterli incelikte sınanamayan bir kodun iyi tasarlanmamış olduğudur.
- Kodlamanın ardından da birim sınamaları yazılabilir; bu white box testing yaklaşımına girer.

Hatalar ne kadar erken bulunursa o kadar iyidir; geliştirme süreci ilerledikçe düzeltmenin zorluğu ve maliyeti artar. TDD uygulanmasa bile bir kısım kodu yazıp sınamak, sonra devam etmek biçiminde küçük adımlar atılabilir.

5.2 Yerine geçen kod

Tek bir sınıf bütün sorumluluklarını tek başına yürütmez; bazıları için başka sınıflara ihtiyaç duyar. Bu yalnız sınama için değil kodlama için de geçerlidir: bir sınıftan diğerine ok varsa önce o diğer sınıfın kodlanması gerekir. Büyük bir tasarım onlarca programcıya dağıtıldığında, ihtiyaç duyulan kısım henüz kodlanmadıysa programcı boş beklemez. İhtiyaç duyulan sınıfın yerine geçecek kod yazılır (ekrandan, 00:30:00):

- Vekil, sahte, yalancı kod/sınıf; İngilizcede stub, dummy, surrogate, proxy.
- Vekil, yalnız ihtiyaç duyulan sınıflar gerçeklenene dek kullanılır.

Hoca bu adlar arasındaki nüanslara hakim olmadığını, bunun yazılım sınama dersinin konusu olduğunu söyledi. Bu sınıflar mümkün olduğunca iskelet kod olarak yazılır: yalnız derleme hatası vermeyecek kadar. Örneğin ayrıntılı bir hesaplama yapıp `true` ya da `false` döndürecek bir metot doğrudan `return true;` der; tam sayı döndürecek bir metot doğrudan `return 1;` ya da başka bir sabit döndürür.

5.3 JUnit hakkında

Sınama masraflı ve kolay olmayan bir iştir; çabayı hafifletecek araçlara ihtiyaç vardır. Birim sınamaları için xUnit ailesi vardır: X yerine dilin adı gelir. JUnit, Java ile yazılmış kodun birim sınamaları için bir çerçeve programdır; diğer diller için de sürümleri vardır, örneğin C# için csUnit (ekrandan, 00:30:00). IDE desteği:

- Eclipse: IDE ile hazır gelir; projeye “build path → add libraries” ile eklenir.
- NetBeans: IDE ile hazır gelir.

IDE’de yoksa JUnit’in sitesinden istenen sürüm indirilip build path’e eklenir.

6. JUnit 3.x ile Test Case Hazırlamak

Hoca 3.x ve 4.x sürümlerinden bahsetti ve ağırlıklı olarak 3.x ile çalışacaklarını söyledi; ihtiyaçları karşılayan en temel sürüm odur.

6.1 Yapı

Slayttaki şema (ekrandan, 00:40:00):

```
paket junit.framework:
  TestCase      # setup()
  Assert        + assertEquals(beklenen, sonuc : Object)
                + assertNotEquals(Object, Object)
```

```
                + assertTrue(boolean)
                + assertFalse(boolean)
                + assertNull(Object)
                + assertNotNull(Object)
  (TestCase ---> Assert : kullanma)

paket myPackage:
  AppClass1, AppClass2
  TestAppClass1 (+ testXXX())  --|> TestCase,  ---> AppClass1
  TestAppClass2 (+ testYYY())  --|> TestCase,  ---> AppClass2
  TestAll (+ suite() : Test)   ---> TestAppClass1, TestAppClass2
```

Slayttaki notlar: her sınıfın birim testi için ayrı sınıfta ayrı test case'ler hazırlanır, ardından bunlar bir test suite altında toplanır. `TestAll` sınıfını yazmak zorunlu değildir; `TestAppClass` sınıfları tek tek de çalıştırılabilir. `TestAll.suite()` metodu statiktir. `TestCase.setup` metodu her `testXXX()` için yeniden çalışır.

6.2 Kurallar

- `AppClass1`'i sınamak için `TestAppClass1` sınıfı yazılır. Hoca bu sürümde sınama sınıfının adının `test` ile başlamasının zorunlu olduğunu söyledi.
- `TestAppClass1`, `junit.framework` paketindeki `TestCase` sınıfından kalıtımla türetilir.
- Sınama sınıfındaki, adı `test` ile başlaması zorunlu olan her `testXXX()` metodu bir test case'tir.
- Her `testXXX()` metodunda bir `assert...` komutu çağrılır. Tercihen bir test metodunda bir assert kullanmak en doğrusudur.

  Ders sonrası not: Hoca JUnit 3'te sınama sınıfının adının `Test` ile başlamasının zorunlu olduğunu söyledi. JUnit 3 çerçevesi büyük olasılıkla yalnız metot adlarının `test` ile başlamasını ve sınıfın `TestCase`'ten türemesini zorunlu tutar; sınıf adındaki `Test` öneki bir adlandırma geleneğidir (bazı derleme araçları sınama sınıflarını bu adla bulur).

6.3 Assert metotları

Sınamada genelde bir metoda belirli bir girdi verildiğinde alınması gereken doğru bir yanıt vardır. Bu yanıtın alınıp alınmadığı uygun `assert` metoduyla denetlenir. Bütün `assert` metotları statiktir.

- `assertEquals(beklenen, sonuc)`: örneğin bir kayıt araması yapıldı; beklenen kayıt ile dönen kayıt aynıysa test başarılıdır.
- `assertNotEquals`: ters mantık; iki nesne aynıysa test başarısızdır.
- `assertTrue(boolean)` ve `assertFalse(boolean)`: elde edilen mantıksal ifadenin doğru ya da yanlış olması beklenir.
- `assertNull(Object)` ve `assertNotNull(Object)`: nesnenin `null` olması (ya da olmaması) beklenir.

Hoca bunların hepsini ezberlemenin gerekmediğini gösterdi: `assertNull(x)` yerine `assertTrue(x == null)` yazmak aynı kapıya çıkar; `assertEquals` yerine de `assertTrue(beklenen == sonuc)` biçimi kullanılabilir.

> **Ders sonrası not:** Hoca `assertEquals`'ın iki parametre "aynı nesne" ise başarılı olduğunu söyledi. `assertEquals` aslında büyük olasılıkla nesnelerin `equals()` metoduyla eşitliğine bakar; aynı nesne (referans eşitliği) denetimi `assertSame` ile yapılır. Aynı biçimde `assertTrue(beklenen == sonuc)` nesnelerde referans karşılaştırması yapar ve iki eşit ama ayrı `String` için başarısız olabilir.

6.4 setUp ve test suite

Bir nesne sorumluluğu tek başına yürütemediği için ya da test için bazı nesnelerin oluşturulup aralarındaki ilişkilerin kurulması gerektiği için, sınama ortamı hazırlanmalı, "sahne kurulmalıdır". Bu iş `TestCase` sınıfının protected `setUp()` metodu ezilerek yapılır. Her `testXXX()` me-

todundan önce `setUp()` yeniden çalışır; böylece her birim test sıfırdan hazırlanmış, ortak ve temiz bir durumla başlar.

Sınama sınıfları istenirse bir test suite altında toplanır. `suite()` metodu yazılarak istenen sınama sınıfları bir grup altında toplanır ve tek tıklamayla hepsi çalıştırılır. Aksi halde sınıflar tek tek çalıştırılır. Metotlar tek tek çalıştırılmaz: bir sınama sınıfı çalıştırılınca içindeki bütün `testXXX()` metotları sırayla çalışır.

7. JUnit 4.x Gelişmeleri ve Annotation Kavramı

7.1 Gelişmeler

Slayt (ekrandan, 00:42:30): geriye doğru uyumluluk korunmakla birlikte JUnit 4 ile annotation desteği gelmiştir (büyük/küçük harf duyarlılığı vardır).

- Sınama sınıflarının `TestCase`'ten türetilmesi gerekmez, ama şu satırlar eklenir:

```
import static org.junit.Assert.*;
import org.junit.*;
```

- Sınama metotlarının adını `test` ile başlatmak gerekmez; metodun başına `@Test` koymak yeter.
- `setUp` adlı metodun yerine `@Before` annotation'ı gelmiştir:

```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }
```

- Exception tanımlanan yazılımlarda atılması gereken exception'ların gerçekten ortaya çıkıp çıkmadığı sınanabilir:

```
@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
}
```

Hocaya göre 3.x'te en çok hissedilen eksiklik exception desteğinin olmamasıdır: runtime ya da checked bir exception oluşunca akış kesilir. Oysa bir işin doğru çalışıp çalışmadığını anlamak için bir exception'ın fırlatılıp fırlatılmadığını sınamak gerekebilir. 4.x bunu `expected` ile destekler: `SomeException` sizin yazdığınız bir exception'dır ve test, bu exception fırlatılırsa başarılı sayılır. Checked exception ise metot imzasına `throws SomeException` yazılmalıdır. JUnit akışı kesmeden bir sonraki birim teste geçer. Ders örneklerinde exception olmadığı için 3.x yeterlidir.

Hoca isimlendirme zorunluluğu kalksa da sınama sınıflarının adını `Test` ile başlatmayı iyi bir sınama alışkanlığı olarak gördüğünü söyledi.

7.2 Annotation nedir?

Annotation'lar `@` karakteriyle başlar ve kaynak kod üzerinde çalışan başka uygulamalara verilen işaretlerdir. Bu örnekte o uygulama JUnit çerçevesidir. Başka bir örnek: Java derleyicisinin verdiği "değişken tanımlanmış ama kullanılmamış" gibi bir uyarıyı kapatmak isteyebilirsiniz. Bunu IDE ayarlarından yapabileceğiniz gibi `@SuppressWarnings("...")` annotation'ı ile de yapabilirsiniz; parantez ve tırnak içine uyarıyı tanımlayan anahtar kelime yazılır. Böylece JDK'nın derleme sürecine müdahale etmiş olursunuz. JUnit annotation'larıyla da JUnit'in test case belirleme ve çalıştırma akışına müdahale edersiniz.

> **Ders sonrası not:** Hoca bir yerde JUnit'in "`@Before` gördüğü her metodu bir test case olarak algıladığını" söyledi (00:43:12). Slayta göre test case'i `@Test` belirler; `@Before` ise her testten önce çalışacak hazırlık metodunu işaretler.

8. Video Kiralama Örneği

8.1 Örneğin yeri

Kod kusurları ve refactoring eylemleri baştan sona tek bir uygulama üzerinde yürütülür. Bu, Fowler'ın kitabında kullandığı örnektir. Fowler kitabında önce bu uygulamayı verip "şurası şu nedenle iyi değil, şöyle düzeltelim" diyerek bütüncül bir refactoring süreci yürütür, sonra kuralları anlatır. Hoca ise önce teorik bilgiyi, sonra pratiği göstermeyi tercih ettiğini söyledi: önce kod kusurlarını anlatacak, ardından bütüncül refactoring örneğini adım adım yapacak.

Örnek video kaset kiralamadır. Hoca örneğin eski olduğunu, ama izle-öde içerikler, uydu aboneliklerindeki ek film paketleri ve tek tek satılan maç yayınlarıyla güncelliğini farklı biçimlerde koruduğunu söyledi.

8.2 Sınıflar

Bilgi sisteminde müşterilerin ve kiralamaların kaydı tutulur (ekrandan, 00:50:00):

```
Movie   [- _priceCode : int]
   ^ 1
   | 1
Rental  [- _daysRented : int]
   ^ *   (hocanın anlatımıyla: bir müşterinin birden fazla kirası)
   | 1
Customer [+ statement()]
```

- `Customer` sınıfının `statement()` metodu müşteriye makbuz basar: hangi kaseti kaç gün kiraladığını, kira bedellerini, toplam borcu ve kazanılan toplam bonus (hediye) puanı.
- Film türü `Movie`'deki fiyat kodu (`_priceCode`) alanında saklanır. Üç tür vardır: yeni çıkan filmler (new release), normal filmler (regular) ve çocuk filmleri (childrens).
- Müşteri film kiralayınca ilişki `Rental` sınıfıyla kurulur. Bir müşterinin çok sayıda `Rental` kaydı olabilir; bir `Rental` yalnız bir filmle ilişkilidir. `_daysRented` filmin kaç günlüğüne kiralandığını tutar.

Neden `Rental` gerekli? Müşteri ile film doğrudan ilişkilendirilseydi, film başka bir müşteriye kiralanınca önceki müşterinin bilgisi silinirdi; müşteri iade ettikten sonra kasetler başkalarına kiralanınca önceki kiraların bilgisi kaybolurdu. Hoca iyi tasarımı şöyle tanımladı: problem hangi düzeyde parçalara ayrılabiliyorsa o düzeyde bileşenleri ayırabilen tasarım. Müşteriler, filmler ve kira işlemleri vardır; kira işlemi de bir problem bileşenidir.

8.3 statement() akışı

Sıralama şemasına göre (ekrandan, 00:50:00) `statement()` bütün kiralar için döner; tek kira için yapılan hesaplar her döngüde biriktirilir:

```
Kullanıcı -> aCustomer : statement()
  *[for all rentals]
    aCustomer -> aRental : getMovie()      -> aMovie
    aCustomer -> aMovie  : getPriceCode()
    aCustomer -> aRental : getDaysRented() => tek kira için ücret hesabı tamamlanır
    aCustomer -> aMovie  : getPriceCode()
    aCustomer -> aRental : getDaysRented() => tek kira için bonus hesabı tamamlanır
    aCustomer -> aMovie  : getTitle()      => tek kira için makbuz satırı basılır
  biriken bilgiyle makbuz basımı tamamlanır
```

Hoca normalde geri dönüş oklarını göstermekte titiz olduğunu, şemada yer kalmadığı için bazılarını göstermediğini söyledi. Kaynak kod ders notlarında `refactoring` paketinin altında, `fowler_00` paketindedir.

8.4 Kod

`Customer` sınıfında kiralar bir generic `Vector<Rental>` içinde tutulur. Üye alanların `_` ile adlandırılması C++ gibi dillerden gelen bir alışkanlıktır. Burada kullanılmasının nedeni, Fowler'ın

kitabının siyah beyaz basılmış olması ve kodun önceki ve sonraki hallerinin yan yana gösterilmesidir; siyah beyaz kodda hangi kelimenin üye alan, hangisinin parametre olduğunu anlamak zordur. Hoca bunun "öyle kod yazın" demek olmadığını, Java genelinde bu geleneğin bulunmadığını, IDE'nin renklendirmenin zaten bunu gösterdiğini vurguladı.

`addRental` listeye bir kira ekler, `getName` adı döndürür. `statement()` epey uzundur ve birçok eleştiri alacaktır. İlk eleştiri: tasarım kalıplarında söylendiği gibi, bir yerde kendinizi switch-case kullanırken görüyorsanız kalıtım ve çok biçimlilik içeren daha iyi bir tasarım gerekir.

Ekrandaki kod (ekrandan, 00:57:40 ve 01:11:45):

```
int frequentRenterPoints = 0;
Enumeration<Rental> rentals = _rentals.elements();
String result = "Rental Record for " + getName() + "\n";
while (rentals.hasMoreElements()) {
    double thisAmount = 0;
    Rental each = rentals.nextElement();
    //determine amounts for each line
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2.0;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3.0;
        break;
    case Movie.CHILDRENS:
        thisAmount += 1.5;
        if (each.getDaysRented() > 3)
            thisAmount += (each.getDaysRented() - 3) * 1.5;
        break;
    }
    // add frequent renter points
    frequentRenterPoints ++;
    // add bonus for a two day new release rental
    ...
```

İş kuralları hocanın anlatımıyla:

- Normal film: 2 lira ödenir ve 2 gün kiralama hakkı verir; 2 günü geçen her gün için 1.5 lira daha ödenir.
- Yeni film: her gün için 3 lira.
- Çocuk filmi: 1.5 lira ile 3 gün kiralama hakkı; 3 günü geçen her gün için 1.5 lira.
- Her kira 1 puan kazandırır; yeni film belirli süreden uzun kiralanırsa 1 ek puan.

Hoca ilk anlatımda kodda dolar yazdığını ama "lira" demenin daha kolay olduğunu söyledi. Döngü her kira için makbuza "şu filmden şu kadar borç" satırını ekler ve toplamı biriktirir; sonunda toplam borç ve kazanılan puan yazılır.

`Rental` sınıfı basittir. `Movie` sınıfındaki tek ilginçlik, film türlerinin `public static final` sabitlerle modellenmesidir (ekrandan, 01:00:00):

```
public class Movie {
    public static final int CHILDRENS = 2;
    public static final int REGULAR = 0;
    public static final int NEW_RELEASE = 1;
    private String _title;
    private int _priceCode;
    public Movie(String title, int priceCode) {
        _title = title;
        _priceCode = priceCode;
    }
    public int getPriceCode() { return _priceCode; }
```

```
    public void setPriceCode(int arg) { _priceCode = arg; }
    public String getTitle() { return _title; }
    ...
}
```

Hoca burada enum bile kullanılmadığını eleştirdi ve bunun ileride bir kod kusuru olarak adlandırılacağını söyledi.

9. Video Kiralama Örneğinde JUnit Uygulaması

9.1 Proje düzeni

Hoca Eclipse projesinde `refactoring` ve `testing` adlı ayrı kaynak klasörleri oluşturdu. Her iki klasörde de aynı adlı paket (`fowler_00`) vardır; fiziksel olarak farklı klasörlerde olsalar da aynı paket sayılırlar. Bunun iki nedeni var:

- Üretim kodu ile sınama kodunu ayrı klasörlerde tutmak; böylece JAR dosyası paketlenirken daha az uğraşılır.
- Sınıfların aynı pakette kalması; böylece ek import gerekmez ve paket düzeyinde erişim gerekiyorsa kolaylık sağlar.

9.2 TestMovie

`TestMovie`, JUnit 3 sürümüne göre yazıldı: `junit.framework.TestCase` import edilir ve sınıf ondan türetilir. Bir tane test metodu vardır. Normalde bir test metodunda bir assert önerilir; birden fazla olursa testin başarılı sayılması için hepsinin başarılı olması gerekir. Hoca örnekte birkaç assert'i tek metoda koyduğunu, aslında bunları ayrı test metotlarına koymanın daha iyi olacağını söyledi.

Her `assert...` metodunun, başına başarısızlıkta gösterilecek açıklamayı alan bir `String` parametre eklenmiş aşırı yüklenmiş sürümü vardır. Örnekte üç film oluşturulur: The Matrix normal, Monsters, Inc. çocuk, Surrogates yeni film. Farklı türdeki filmlerin fiyat kodlarının eşit olmaması beklenir: bu `assertTrue("bunlar farklı tür filmlerdir", ... != ...)` biçiminde yazılır. Bir varyasyonda eşitlik ifadesi yazılıp `assertFalse` kullanılır. Hoca mümkün olduğunca farklı varyasyon göstermek istediğini söyledi.

9.3 TestCustomer

`TestCustomer` de `TestCase`'ten türer. Bir müşteri ve dört film üye alan olarak tanımlanır. Nesneler arasındaki ilişkiler teste göre değiştiği için `setUp()` yalnız bu nesneleri ilklendirir; ilişkiler test metotlarında kurulur. Testler (ekrandan, 01:11:45):

- `testGetName`: müşteri adında Türkçe karakter vardır; adın doğru döndüğü ve Türkçe karakterlerin desteklendiği `assertEquals` ile sınanır. Örnek müşteri adı olarak hoca kendi adını kullandı.
- `testStatementWhenEmpty`: henüz film kiralamamış müşterinin makbuzu boş olmalıdır: film listelenmez, sıfır borç ve sıfır puan yazılır.
- `testStatementWithMoviesLongRent`: her türden film, ek ücret gerektirecek uzunlukta kiralanır.
- `testStatementWithMoviesShortRent`: aynı şey kısa süreli kiralarla yapılır.
- `testNewReleaseRentalBonus`: bir yeni film ve bir film daha kiralanıp bonusun doğru hesaplandığına bakılır.

Her testte müşterinin `statement()` sonucu alınır, beklenen makbuz metni elle oluşturulur ve ikisi `assertEquals(beklenen, sonuc)` ile karşılaştırılır.

> **Hocanın vurgusu:** Beklenen sonuç elle yazılmalıdır. Hatalı bir otomatik yöntemle üretilirse hata keşfedilemez. Test metotlarına da `test1` gibi anlamsız değil, neyi sınadığını açıkça anlatan adlar verilmelidir; gerekirse uzun olsun.

Bonus testi (ekrandan, 01:10:00):

```
public void testNewReleaseRentalBonus( ) {
    musteri.addRental( new Rental(surrogate, 2) );
```

```
    musteri.addRental( new Rental(terminator, 1) );
    String sonuc = musteri.statement();
    String beklenen = "Rental Record for <müşteri adı>\n";
    beklenen += "\tSurrogates\t6.0\n";
    beklenen += "\tTerminator Salvation\t3.0\n";
    beklenen += "Amount owed is 9.0\n";
    beklenen += "You earned 3 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

Ekranda müşteri nesnesinin değişken adı hocanın adıdır; burada `musteri` diye yazıldı. Hesap: Surrogates yeni film, $2 \cdot 3.0 = 6.0$. Terminator Salvation 1 günde 3.0 tuttuğuna göre o da yeni film olarak tanımlanmıştır ($1 \cdot 3.0$; normal film olsaydı 2.0 olurdu) (ara adım eklendi). Toplam $6.0 + 3.0 = 9.0$. Puan: iki kira için 2 puan ve 2 günlük yeni film kirası için 1 ek puan, toplam 3.

> **Ders sonrası not:** Hoca ek puanın "yeni film iki günden daha uzun süre kiralanırsa" verildiğini söyledi. Ekrandaki test 2 günlük yeni film kirasında ek puan beklediğine ve koddaki yorum "two day new release rental" dediğine göre, koşul büyük olasılıkla "1 günden fazla" yani en az 2 gündür.

Hoca bir sorunu da gösterdi: aslında yalnız bonusun doğru kazanılıp kazanılmadığını sınamak istiyor, ama tasarım buna izin vermiyor; bonusun hesaplandığı ayrı bir yer yok. Bu yüzden bonus ancak makbuzun tamamı üzerinden sınanabiliyor. En azından bu testler sonraki refactoring adımlarında işi kolaylaştıracak.

9.4 Çalıştırma ve hata canlandırması

Sınama sınıflarında `main` metodu yoktur ama `testXXX()` metotları vardır; sınıfa sağ tıklanıp "Run As → JUnit Test" seçilir. Eclipse JUnit perspektifine geçer. `TestMovie` için 1 test çalışır ve yeşil çubuk görülür. `TestCustomer` için 5 test case çalışır ve hepsi geçer.

Sonra hoca bilerek bir hata yaptı: müşteriyi yanlış anladığını varsayıp çocuk filmi taban ücretini 1.5 yerine 1.0 yazdı (`thisAmount += 1.0;`). Testler yeniden çalışınca kırmızı çubuk çıktı: 5 testten 2'si (kısa ve uzun kira testleri) başarısız (ekrandan, 01:11:45). Hata izinde `junit.framework.ComparisonFailure` görülür: beklenen metin ve elde edilen metin verilir, farklılaşmanın başladığı ve bittiği yer köşeli parantezle işaretlenir:

```
expected:<...0
Monsters, Inc. 1.[5
Surrogates 3.0
Amount owed is 6.5]
You earned 3 freque...> but was:<...0
Monsters, Inc. 1.[0
Surrogates 3.0
Amount owed is 6.0]
You earned 3 freque...>
```

Kısa kira testinde beklenen toplam 6.5'tir; çocuk filmi satırı 1.0 olunca toplam $6.5 - 0.5 = 6.0$ olur. Görünen satırlardan diğer film satırının 2.0 olduğu çıkar: $6.5 - 1.5 - 3.0 = 2.0$ (ara adım eklendi). Hoca değeri yeniden 1.5 yapınca bütün testler geçti ve yeşil çubuk göründü. Sınamalar uzun sürüyorsa yalnız başarısız olanlar yeniden koşturulabilir.

9.5 TestAll

`TestCustomer` ve `TestMovie`'yi ayrı ayrı çalıştırmak yerine bir suite yazılır (ekrandan, 01:13:20):

```
package fowler_00;

import junit.framework.*;

public class TestAll {
    public static Test suite( ) {
        TestSuite suite = new TestSuite("Bütün testler");
```

```
        suite.addTestSuite(TestCustomer.class); //reflection API
        suite.addTestSuite(TestMovie.class);
        return suite;
    }
}
```

`TestAll` çalıştırılınca 6 test koşar: "Bütün testler" altında `TestCustomer`'ın 5 ve `TestMovie`'nin 1 testi görünür. `.class` üyesi Java'nın Reflection API'sinin parçasıdır; hoca bunu code introspection diye de andı.

> **Ek bilgi (derste anlatılmadı):** Reflection, bir programın çalışma anında kendi sınıflarını, metotlarını ve alanlarını sorgulayabilmesidir. JUnit 3, `TestCase` alt sınıfındaki `test` ile başlayan public metotları büyük olasılıkla bu yolla bulup çalıştırır; `addTestSuite(X.class)` de bu sınıf nesnesini kullanır.

10. Duplicated Code ve Extract Method

Aradan sonra akış şöyle kuruldu: önce bir kod kusuru tarif edilir, sonra onu gideren bir ya da daha fazla refactoring eylemi tanıtılır. Tasarım kalıplarında olduğu gibi literatürdeki özgün adlar kullanılır; slayt başlığında özgün ad, altında Türkçe karşılık denemesi yer alır.

10.1 Duplicated Code (yinelenen kod)

Aynı kodun ya da küçük farklılıklarla çok benzer kod yapısının birden fazla yerde tekrarlanmasıdır. Dezavantajı: kopyalanmış kodun işlevinde bir değişiklik gerektiğinde, kodun yapıştırıldığı her yerde aynı değişikliği yapmak gerekir; bu epey angaryadır. Önerilen refactoring eylemlerinden biri Extract Method'dur; başkaları da vardır.

10.2 Extract Method (metot çıkartma)

Slayt (ekrandan, 01:40:00):

- Bir metot içindeki kodun bir kısmını yeni bir metot olarak belirlemeye dayanır.
- Bu yapılandırma birçok kod kusurunu düzeltir (örneğin Long Method).
- Oluşturulan yeni metoda amacını açıkça belirten, isabetli bir ad verilmelidir. Böylece alt düzey ayrıntılarla uğraşmadan kodun anlaşılabilirliği artar; hem metodun çıkarıldığı yer hem çıkarılan metot daha kolay anlaşılır.

Örnek:

```
/*** ÖNCE: ***/
void printOwing(double amount) {
    printBanner();
    //print details
    System.out.println("name:" + _name);
    System.out.println("amount" + amount);
}

/*** SONRA: ***/
void printOwing(double amount) {
    printBanner();
    printDetails(amount);
}
void printDetails(double amount) {
    System.out.println("name:" + _name);
    System.out.println("amount" + amount);
}
```

Önceki halde iki ayrı iş yapılıyor: başlığın yazılması ve ayrıntıların yazılması; yorum satırı da bunu söylüyor. Ayrıntı kısmı kesilip `printDetails` adlı metoda çıkarılır; `printOwing`'e gelen `amount` yeni metoda parametre olarak aktarılır.

Sonraki halin avantajları:

- `printOwing`'in ne yaptığını anlamak için yalnız `printBanner` ve `printDetails` çağrılarına bakmak yeter. Önceki halde hem kodu hem yorumları incelemek gerekiyordu.
- Ayrıntılardan biri gerekiyorsa o metoda gidilir (Eclipse'de Ctrl ve sol tık). Nereyi anlamak ya da düzenlemek gerekiyorsa odaklanmış biçimde orası anlaşılır ve düzenlenir.

  Hocanın vurgusu: Sonraki halde tek satır yorum yok. Hocalarımız "mümkün olduğunca yorum yazın" derdi; refactoring bunu bir adım ileri taşır: kodu öyle yazın ki yoruma mümkün olduğunca gerek kalmasın.

11. Long Method ve İkinci Extract Method Örneği

11.1 Long Method (uzun metot)

Bir metodun uzunluğu arttıkça anlaşılması zorlaşır ve herhangi bir gereksinim değişikliğinden etkilenme olasılığı artar. Uzun metotta fazla iş yapılır ve bu işlerin birbiriyle ilgisiz olma riski artar; başka bir deyişle birden fazla ilgi alanına ya da sorumluluğa bulaşma olasılığı artar, bu da uyumu (cohesion) düşürür. Bir sınıfta ne kadar farklı türden iş yapılırsa uyumu o kadar düşer; bu metot düzeyine de indirilebilir. İlk çözüm Extract Method'dur: ortak amaca hizmet eden kod parçaları kendi metotlarına toplanır.

11.2 printOwing örneği

Slayt (ekrandan, 01:42:10):

```
/*** ÖNCE: ***/
void printOwing( ) {
    Enumeration e = _orders.elements();
    double outstanding = 0.0;

    // print banner
    System.out.println("******************");
    System.out.println("* Customer Owes *");
    System.out.println("******************");

    // calculate outstanding
    while (e.hasMoreElements()) {
        Order each = (Order) e.nextElement();
        outstanding += each.getAmount();
    }

    //print details
    System.out.println ("name:" + _name);
    System.out.println ("amount" + outstanding);
}

/*** SONRA: ***/
void printOwing( ) {
    printBanner();
    printDetails(getOutstanding());
}

void printBanner() {
    System.out.println("******************");
    System.out.println("* Customer Owes *");
    System.out.println("******************");
}

double getOutstanding() {
    double outstanding = 0.0;
    for (Order each : _orders)
        outstanding += each.getAmount();
```

```
    /* for-each ile döngü kısaldı! */
    return outstanding;
}

void printDetails (double outstanding) {
    System.out.println ("name:" + _name);
    System.out.println ("amount" + outstanding);
}
```

Metotta üç ayrı iş vardır: başlık basmak, ödenmemiş borcu (outstanding) hesaplamak ve ayrıntıları basmak. Her yorum bloğundan bir metot çıkarılır; yorumlar olmasaydı bile işler anlaşılırdı. `printOwing` iki satıra iner.

Metot çıkarırken kullanılan değişkenlere ve parametrelere dikkat etmek gerekir. `_orders` ve `_name` üye alan olduğu için sorun yoktur. Ama ayrıntı kısmının son satırı, `printOwing` içinde tanımlanmış geçici `outstanding` değişkenini kullanır. `outstanding` döngüde hesaplanır ve döngü bitince hazır olur. Çözüm: döngü `getOutstanding()` metoduna alınır (bu arada for-each ile kısaltılır); metot hesapladığı değeri döndürür, `printOwing` de bu değeri `printDetails`'e parametre olarak verir. Böylece bir taşla iki kuş vurulur: gerekli bilgi `printDetails`'e ulaşır ve `printOwing`'deki geçici değişken ortadan kalkar. `getOutstanding` içindeki geçici değişkenden kaçınılamaz; kümülatif toplam için bir biriktirme değişkeni şarttır.

11.3 İtirazlar

Hoca öğrencileri itiraz etmeye davet etti ve bu dersi verdikçe biriken itirazları kendisi dile getirdi.

Soru: Bir metot dörde bölündü. İnsan aynı anda sınırlı sayıda parçayı aklında tutabilir; dört metodu birden aklımda tutamayabilirim. **Cevap:** Hoca bu sınırı "5 artı eksi 2" diye verdi: yetenekli ya da o gün iyi durumdaki biri 7 birimle, kötü durumdaki biri 3 birimle çalışabilir. Ama hepsini aynı anda bilmek gerekmez. İlgilenilen sorun ayrıntıların basılmasıysa `printBanner`'a hiç bakılmaz; yalnız `printDetails` ve parametresini üreten `getOutstanding` incelenir. Bu iki metot rahatça akılda tutulur.

> **Ders sonrası not:** Hoca kısa süreli bellek sınırını "5 artı eksi 2" diye verdi. Literatürde bilinen değer büyük olasılıkla George Miller'ın "7 artı eksi 2" sınırıdır (5 ile 9 arası).

Soru: Her metot çağrısında bağlam değiştirme (context switching) olur: çağıran metodun durum bilgisi yığına itilir, yeni metot çağrılır. Bunun maliyeti yok mu, kodun başarımı düşmez mi? **Cevap:** Modern diller ve derleyiciler, yorumlayıcıyla ya da sanal makineyle çalışan ortamlar bile, bu yükü onlarca yıldır en aza indirmiştir. Bu yük 30-40 yıl önceki dil sürümlerinde ve bilgisayarlarda hissediliyordu; artık fark etmez. Nesneye dayalı yaklaşımın "böl, parçala, yönet" mantığı çekinmeden kullanılabilir.

11.4 Toparlama

Nesneye dayalı yaklaşım, büyük sınıflar ve az sayıda nesne yerine daha çok sayıda küçük sınıf ve nesne kullanmayı önerir. Böylece kodun programın çeşitli yerlerinde paylaşımı kolaylaşır; çıkarılan küçük metotlar gereken her yerden çağrılabilir. Hoca paylaşımı (aynı kod içinde) yeniden kullanımdan (farklı projelerde) ayırdı. Öneri: uzun ve çok iş yapan metotlar yerine kısa ve adıyla kendini açıklayan ("ismiyle müsemma") metotlar.

12. Yinelenen Kodun Yerini Bulmak: Pull Up Method ve Form Template Method

Yinelenen kodun nerede ve nasıl yinelendiğine göre farklı refactoring eylemleri vardır (ekrandan, 01:50:00):

- **Pull Up Method:** Yinelenen kod kalıtım ağacındaki kardeş ya da kuzen sınıflara yayılmışsa, oluşturulan metot ortak üst sınıfa çekilir. Adında "pull up" ya da "pull down" görülen bir eylem, bir kalıtım hiyerarşisinde yukarı ya da aşağı hareket demektir.

- **Form Template Method:** Kod aynı değil ama benzer ana adımlar varsa, GoF Template Method tasarım kalıbı kullanılarak kardeş/kuzen sınıflarda düzenleme yapılır. Hoca burada tasarım kalıplarının önemli bir yararına yeniden tanık olunduğunu söyledi: ekip içinde ortak bir sözlük oluşturup iletişimi verimli kılmak. Template Method bilindiği için bu refactoring'in ne yaptığı hemen anlaşılır.
- **İlişkisiz sınıflar:** Yinelenen kod aralarında sahiplik ya da kalıtım ilişkisi olmayan sınıflardaysa büyük olasılıkla tasarım hepten kötüdür; sorumluluklar doğru dağıtılmamıştır. Daha büyük bir revizyon daha doğru olabilir. Bu yapılamıyorsa metot onu kullanan sınıflardan birine atanır, diğerleri bu sınıfı kullanır; ya da metot yeni oluşturulan bir sınıfa atanır ve diğerleri bu sınıfı kullanır.

Özetle Extract Method ile oluşturulan yeni metoda en uygun yer aranır. Yerleştirilen yere göre eylemin adı Extract Method ya da Pull Up Method olur. Extract Method çıkarılan kodun aynı sınıfta mı farklı sınıfta mı kaldığına bakmaz; yalnız kalıtım hiyerarşisinde hareket varsa Pull Up ya da Pull Down adı kullanılır.

12.1 Pull Up Method örnekleri

Fowler'ın kitabından alınan şemada hem `Salesman` hem `Engineer` sınıfında `getName()` vardır; bu ortak metot en iyi ortak üst sınıf `Employee`'de tanımlanır. Hoca şemanın kitaptan alındığını, üye alanların görünürlüğünün ve türlerinin (ör. `String`) gösterilmediğini, öğrencilerin böyle yapmaması gerektiğini söyledi; public üye alanları private yapmaya dikkat etmeyen öğrenciler olduğunu da hatırlattı.

Özel durum (ekrandan, 01:54:40):

```
ÖNCE:
Customer [lastBillDate | addBill(dat: Date, amount: double)]
  <|-- Regular Customer   [createBill(Date), chargeFor(start: Date, end: Date)]
  <|-- Preferred Customer [createBill(Date), chargeFor(start: Date, end: Date)]
  (not: iki chargeFor "çok farklı kodlanmışlar")

SONRA:
Customer [lastBillDate | addBill(dat: Date, amount: double),
          createBill(Date), chargeFor(start: Date, end: Date) {soyut}]
  <|-- Regular Customer   [chargeFor(start: Date, end: Date)]
  <|-- Preferred Customer [chargeFor(start: Date, end: Date)]
  (not: chargeFor metodu üst sınıfta soyut tanımlandı)
```

`chargeFor` başlangıç ve bitiş tarihleri arasında faturalama yapar, ama iki alt sınıfta çok farklı kodlanmıştır; benzerlik yok denecek kadar azdır. O zaman hiçbir şey yapılamasa bile `chargeFor` üst sınıfta soyut metot olarak tanımlanır. Bütün müşteriler bir listede tutuluyorsa hepsine tek hamlede `chargeFor` çağrılabilir; hangi alt sınıftansa onun metodu çalışır. Ara durumda, yani bazı ortak kodlar bulunuyorsa, slayttaki gibi Template Method tasarım kalıbı kullanılabilir.

13. Decompose Conditional

Long Method'u çözen başka refactoring eylemleri de vardır; birincisi Decompose Conditional'dır. "Conditional", `if` ya da `switch-case` gibi karar verme komutlarıdır. `if` ya da `switch-case` görmek zaten kalıtım ve çok biçimliliğin kullanılması gerektiğinin işaretidir. Karar komutları ve döngüler kodu uzatır ya da ileriki sürümlerde uzama olasılığını artırır. Bu yüzden karmaşık `if` komutları alt parçalara bölünür ve gerekirse uygun `and`, `or`, `not` gibi mantıksal işleçlerle birleştirilir.

Slayt (ekrandan, 02:00:00): aşırı karmaşık `if` komutlarını sadeleştirmek anlaşılabilirliği artırır.

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;

/*** SONRA: ***/
```

```
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);

private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
        date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
    return quantity * _winterRate + _winterServiceCharge;
}
```

Hoca önceki koşulu çözümlemek için bir zaman doğrusu çizdi: `SUMMER_START` ve `SUMMER_END` işaretlendi. Tarih yaz başlangıcından önceyse ya da yaz bitişinden sonraysa, yani "yaz değilse" kış tarifesi uygulanır. Büyük harfli adlar sabittir. Bu kadar basit bir anlamı çıkarmak için bu kadar çaba harcamak gerekmemelidir. Koşulun içi pek sadeleşmez, ama ne yaptığı anlaşıldığı için ona uygun ad verilir: `notSummer(date)`.

Dallar da incelendi: kışın ücret, miktar çarpı kış oranı artı kış servis bedelidir; yazın yalnız miktar çarpı yaz oranıdır. İçinde yalnız iki işleç olsa da bu bir iş kuralıdır ve başlı başına bir iştir; bu yüzden `summerCharge` ve `winterCharge` metotları çıkarılır. Ortak ihtiyaç duyulan parametre `quantity`'dir. Sonuç çok daha kolay anlaşılır; bir ayrıntıyla ilgili sorun varsa ilgili metoda girilir ve diğer ayrıntılarda boğulmadan anlaşılır. Bu da bakımı kolaylaştırır.

Çıkarılan metotları sıradan kullanıcıların kullanmasına gerek olmadığı için hoca bunları `private` işaretledi. Başka nesnelerin de kullanması yararlı olacaksa görünürlük genişletilebilir.

14. Long Parameter List: Introduce Parameter Object ve Preserve Whole Object

14.1 Long Parameter List

Fowler'ın dikkat çektiği bir başka olgu: bir metodun aldığı parametre sayısı çok fazlaysa ve bu parametrelerin en azından bir kısmı başka metotlarda da hep birlikte geçiyorsa, bu parametreler bir varlığın durum bilgisinin bir kısmını ya da tamamını oluşturur. Bu, iyi bir tasarım yapılamadığını gösterir: tasarım, problemin ayrıldığı düzeyde parçalara ayrılmalıydı, ama parçalar dağınık kalmış. Duruma göre Introduce Parameter Object ya da Preserve Whole Object uygulanır.

14.2 Introduce Parameter Object (parametre nesnesi oluşturma)

Slayt (ekrandan, 02:06:40): bir metot çok fazla parametre alıyorsa, uygun parametreleri üye alan olarak yeni bir sınıfta toplayın ve o türden bir nesneyi parametre olarak kullanın.

```
ÖNCE: Customer
  amountInvoicedIn(start: Date, end: Date)
  amountReceivedIn(start: Date, end: Date)
  amountOverdueIn(start: Date, end: Date)

SONRA: Customer
  amountInvoicedIn(DateRange)
  amountReceivedIn(DateRange)
  amountOverdueIn(DateRange)
```

Üç metot, bir zaman aralığında faturalanan, ödenen ve gecikmiş tutarları hesaplar; hepsi başlangıç ve bitiş tarihini alır. O zaman `start` ve `end` üyeli bir `DateRange` sınıfı oluşturulur ve bu sınıfın nesnesi parametre olarak verilir. Hoca, Fowler'ın `DateRange`'i şemada ayrı bir sınıf olarak göstermesinin daha iyi olacağını söyledi. Aynı tarih aralığı başka sınıfların metotlarında da kullanılabilir.

Slayttaki ek noktalar:

- Bu yalnız parametre listesini kısaltmak için yapılmaz; `DateRange` yalnız iki veriyi birleştiren bir kayıt yapısı olarak kalmaz.
- `isWithinRange(Date)` gibi uygun metotlar da içerdiği için Decompose Conditional ya da Extract Method düzenlemesi de yapılmış olur.

Hocaya göre bir tarihin bu aralıkta olup olmadığına göre iş yapan kodlar yüzde 99 olasılıkla zaten vardır: ya gereken her yerde ayrı ayrı yazılmıştır (Duplicated Code), ya da bir statik metotla ortaklaştırılmıştır. Statiklik çok gerekmedikçe nesneye dayalı yaklaşımda tercih edilmez, çünkü her nesne ayrı bir bireydir. `DateRange`, "tarih aralıkta mı", "aralık dışındaysa kaç gün geçmiş", "dolmasına kaç gün var", "başlangıcından bu yana kaç gün geçmiş" gibi metotların en uygun yeridir.

14.3 Preserve Whole Object (tüm nesneyi gönder)

Slayt (ekrandan, 02:10:00): bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine nesnenin kendisini gönderin.

```
/*** ÖNCE: ***/
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);

/*** SONRA: ***/
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

`daysTempRange()` günün sıcaklık aralığını döndürür; adı kısaltılmıştır. Önceki halde en düşük ve en yüksek değer iki geçici değişkende tutulup `withinRange`'e verilir; metot değerlerin planın içinde olup olmadığını `true` ya da `false` olarak döndürür. Sonraki halde nesnenin kendisi tek parametre olarak gönderilir; gerekiyorsa açıp kullanmak `withinRange`'in işidir. Bir taşla iki kuş: hem parametre listesi kısalır hem geçici değişkenler kalkar.

Slayttaki zayıf yönler:

- **1. zayıf yön, bağımlılık artabilir:** parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir (kullanma) bağımlılığı oluşur. Ama kodun geneline bakınca o türden bir üye alan zaten varsa, yani arada zaten sahiplik ilişkisi varsa, ek kullanma ilişkisinin bir etkisi olmaz. Yeni bir ilişki oluşuyorsa bağlaşım (coupling) ölçüleri biraz kötüleşir; kazanç ve kayıp tartılır.
- **2. zayıf yön, geriye doğru uyumluluk kaybedilir ve refactoring tanımı esnetilir:** refactoring dışarıya açılan pencereyi değiştirmemekti, oysa metot imzası değişti. Hoca "bu refactoring yapılmamalı" diyenlere hak verdi. Ama geriye doğru uyumluluk şu an önemli değilse, örneğin çok sayıda test case'i yeniden yazmak gerekmiyorsa, sahada kodun eski sürümleri yoksa ya da bu sürümle birlikte çalışması gereken eski sürümler yoksa, bu zayıflık boşa düşer.
- Bu durumlardan biri bile çok sakıncalı olacaksa bu düzenleme kullanılmaz.

Hoca iletişim yükü konusuna da değindi: iki `int` (4 + 4 = 8 bayt) yerine bütün bir nesne gönderilir. Aynı bellek uzayında bunun önemi yoktur. Parçalar farklı bilgi sistemlerinden sorgulanıyorsa durum farklıdır, ama aynı varlığın üyelerinin farklı adres uzaylarında olma olasılığı düşüktür. Hoca, bankalar gibi finansal sistemlerde böyle durumlar olduğunu duyduğunu söyledi.

> **Hocanın vurgusu:** Zayıflıklar boşa düşüyorsa çekinmeden yapın. Düşmüyorsa kaybettiklerinizi terazinin bir kefesine, kazandıklarınızı öbür kefesine koyun ve hangisi daha önemliyse ona göre karar verin.

15. Replace Temp with Query ve Başarım Tartışması

15.1 Geçici değişkenlere neden "gıcığız"?

Hoca geçici değişkenlerin bir yararı olduğunu kabul etti: bir hesaplama sonucunu geçici olarak saklamak, o hesabı tekrar tekrar yapmayı önler. Ama her geçici değişken kodun uzamasını teşvik eder: "bu sonucu saklıyorum, şu hesapta da kullanayım" derken metoda daha fazla iş yüklenir ve Long Method oluşur; ya da kod benzer yerlere kopyalanır ve Duplicated Code oluşur. Hoca bunu "şeytan dürter" sözüyle anlattı. En iyisi önlemi şimdiden almak ve geçici değişkenlerden mümkün olduğunca kurtulmaktır. Kümülatif toplam gibi başka türlü hesaplanamayan durumlar istisnadır.

Bu iki eylem (Extract Method ve Replace Temp with Query) işe yaramazsa daha karmaşık bir eylem kullanılır: Replace Method with Method Object (Bölüm 16).

15.2 getPrice örneği

Slayt (ekrandan, 02:16:40): bir ifadeyi bir geçici değişkende saklayarak kullanmak yerine, ifadeden metot(lar) çıkarın.

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}

/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

Taban fiyat 1000'den büyükse indirim katsayısı 0.95 (yüzde 5 indirim), değilse 0.98'dir (yüzde 2 indirim). Metot aslında iki iş yapar: taban fiyata göre indirim oranını belirlemek ve taban fiyatı bu oranla çarpmak. Bu yüzden hem Replace Temp with Query hem Long Method düzeltmesi yapılır. Hoca, buna Extract Method da denebileceğini, ama sonuçta geçici değişkenlerden kurtulunduğu için adın Replace Temp with Query olduğunu söyledi. Bütüncül örnekte daha ayrıntılı bir sürümü yapılacak.

Soru: `basePrice`'ı boşuna geçici değişkende tutmadık, değerine göre karar verdik. Şimdi `getPrice` içinde `basePrice()` çağrılıyor, `discountFactor()` içinde de yeniden çağrılıyor; metot iki kez çağrılmış olmuyor mu? **Cevap:** Haklısınız, ama böyle basit bir metodun bir kez daha çağrılması fark etmez. Bu metotlar başka makinelerde çalışan, ağ üzerinden yapılan API çağrılarıysa ve ağ trafiği, zaman ve haberleşme masrafı önemliyse bu refactoring yapılmaz. Diğer bütün durumlarda yapılması önerilir.

Kazanç: kod daha anlaşılır olur ve kopyala yapıştır daha ortaya çıkmadan azalır. Geçici değişkenli bir metodun benzerini başka yerde yazmak gerekince el hemen Ctrl-C, Ctrl-V'ye gider ve Duplicated Code doğar. Hesap sorgu metoduna alınırsa her yerden tek satırla çağrılır ve değişiklik tek yerde yapılır.

15.3 printReceipt örneği: birden çok kullanım

Slayt (ekrandan, 02:20:00): geçici değişken metot içinde birden fazla yerde kullanılıyorsa aynı işlem tekrar tekrar yapılır; bu başarımı daha da düşürür.

```
void printRecipt() { /*** ÖNCE: ***/
    int totalAmount = 0, thisAmount;
```

```
    for( Item item : items ) {
        thisAmount = item.getCharge( );
        System.out.println(item.getName()+" "+thisAmount);
        totalAmount += thisAmount;
    } }

void printRecipt() { /*** SONRA: ***/
    int totalAmount = 0;
    for( Item item : items ) {
        System.out.println(item.getName()+" "+item.getCharge());
        totalAmount += item.getCharge( );
    } }
```

Kümülatif `totalAmount` için yapılacak bir şey yoktur, ama `thisAmount` kaldırılabilir: ona `item.getCharge()` atandıktan sonra değeri değiştirilmeden yalnız okunuyor. O zaman bu ifade `thisAmount`'un geçtiği her yere taşınır.

Burada çağrı sayısı bir yerden iki yere çıkar ve bu artış döngünün içindedir. Örneğin 1000 öğe varsa, eskiden 1000 kez çağrılan metot şimdi 2000 kez çağrılır; hissedilir bir başarım düşüşü olabilir.

Hocanın cevabı: büyük olasılıkla sistemin genelinde başka bir darboğaz vardır ve buradaki kayıp onun yanında genelde küçük kalır. Benzetme Haliç'ten: Haliç Köprüsü'ne 3 şerit ana yol ve 2 şerit yan yol, toplam 5 şerit gelir; köprüde yol 4 şeride düşer (iki şerit orta, iki şerit yan köprü). Orada hep sıkışıklık olur; seyahat süresini belirleyen bu darboğazdır. Yol üzerindeki bir trafik ışığı ise aslında akışı düzenleyerek herkesin daha az zaman harcamasını sağlar. Işıklarla uğraşmak yerine köprüyü genişletmek akışı rahatlatırdı.

> **Hocanın vurgusu:** Yapılan refactoring sistemin mevcut darboğazını aşan bir yük oluşturmuyorsa yapılsın. Yeni bir darboğaz oluşturuyorsa ya da darboğaza yaklaştırıyorsa yapılmasın. Bunu anlamak için profiler denen, programın hangi kısımlarında ne kadar süre harcandığını raporlayan araçlar vardır. Profiler başarımın düştüğünü gösterirse değişiklik geri alınır; refactoring küçük adımlarla yapıldığı için geri almak kolaydır.

16. Replace Method with Method Object

Slayt (ekrandan, 02:27:20):

- Metot öyle uzun ve geçici yerel değişkenleri öyle arap saçı gibi kullanıyor ki Extract Method düzenlemesini kullanamıyorsunuz.
- O zaman yeni bir sınıf oluşturup bütün metodu o sınıfa taşıyın, yerel değişkenleri de o sınıfın üyeleri yapın.
- Ardından bildiğiniz düzenleme eylemlerini daha rahat yürütebilirsiniz.
- Yeni sınıfın adını verirken taşınan metodun adına benzer bir ad verin.

```
ÖNCE:
class Order {
    double price() {
        double primaryBasePrice;
        double secondaryBasePrice;
        double tertiaryBasePrice;
        // long computation;
        ...
}

SONRA:
Order [price()]  - - - kullanır - - ->  PriceCalculator
                                          primaryBasePrice
                                          secondaryBasePrice
                                          tertiaryBasePrice
                                          compute()
```

```
PriceCalculator ---> Order (1)
price() notu: return new PriceCalculator(this).compute()
```

`Order` sınıfındaki `price()` metodunda üç geçici değişken var; uzun bir hesaplamada bu değişkenlere hem değer atanıyor hem değiştiriliyor hem kullanılıyor. Önerilen çözüm, en azından `Order` sınıfını "adam etmek" için karmaşıklığı tek bir noktaya toplamaktır: fiyatı hesaplamak için `PriceCalculator` sınıfı oluşturulur. Geçici değişkenler bu sınıfın üye alanları olur, `price()`'ın gövdesi `compute()` metoduna taşınır. `Order.price()` yalnız bir `PriceCalculator` nesnesi oluşturup `compute()`'u çağırır; yani yalnız çağrı iletme (delegation) yapar.

Eski metot `Order`'ın çeşitli üye alanlarına erişim metotlarıyla ulaşıyordu. Bu verileri tek tek aktarmak yerine Preserve Whole Object uygulanır: `Order` kendisini (`this`) `PriceCalculator`'a verir. Böylece `PriceCalculator`'dan `Order`'a bir sahiplik ilişkisi, `Order`'dan `PriceCalculator`'a kesikli çizgiyle gösterilen bir kullanma ilişkisi oluşur. `compute()` ilk anda düzeltilemese bile en azından `Order` sınıfı "adam edilmiş" olur: istatistikleri iyileşir ve bakımı kolaylaşır.

Örnek Sorular

Soru 1 (derste sözlü, 00:10:49): Font seçme penceresinde 4 bileşen birbirleriyle çift yönlü doğrudan haberleşiyor. Kaç bağlantı vardır? Beşinci bileşen eklenince kaç bağlantı daha gerekir? Mediator kullanılırsa ne değişir?

> **Çözüm (derste verildi):** 4 bileşende 12 bağlantı vardır. Beşinci bileşen önceki 4 bileşenin her biriyle iki yönlü bağlanacağından $4 \cdot 2 = 8$ bağlantı daha gelir. Mediator kullanılınca bileşenler birbirleriyle değil, yalnız merkezi Mediator nesnesiyle haberleşir; yeni bir bileşen yalnız Mediator'a bağlanır.

Soru 2 (ekrandan, 01:11:45): `Customer.statement()` içinde çocuk filmi taban ücreti `thisAmount += 1.5;` yerine yanlışlıkla `thisAmount += 1.0;` yazılmıştır. Kısa kira testinde beklenen makbuz "Monsters, Inc. 1.5", "Surrogates 3.0" ve "Amount owed is 6.5" satırlarını içerir. JUnit hangi sonucu verir?

> **Çözüm (derste verildi):** Test başarısız olur (kırmızı çubuk). `ComparisonFailure` beklenen metinde "Monsters, Inc. 1.[5 … Amount owed is 6.5]", elde edilen metinde "Monsters, Inc. 1.[0 … Amount owed is 6.0]" gösterir; köşeli parantezler farkın başladığı ve bittiği yerdir. Toplam 0.5 eksik çıkar. Uzun kira testi de başarısız olur; 5 testten 2'si kalır.

Soru 3 (derste sözlü, 01:47:40): Extract Method ile bir metodu dört metoda bölmek, her çağrıdaki bağlam değiştirme yükü yüzünden başarımı düşürmez mi?

> **Çözüm (derste verildi):** Modern diller ve çalışma ortamları (sanal makine ve yorumlayıcılar dahil) metot çağrısı yükünü onlarca yıldır en aza indirmiştir. Bu yük ancak 30-40 yıl önceki sistemlerde hissediliyordu. Böl ve yönet mantığı çekinmeden kullanılabilir.

Soru 4 (derste sözlü, 02:17:26): Replace Temp with Query sonrası `getPrice()` örneğinde `basePrice()` iki kez çağrılıyor. Bu refactoring hangi durumda yapılmamalıdır?

> **Çözüm (derste verildi):** Basit bir metodun fazladan bir kez çağrılması fark etmez. Ama çağrılan metot başka makinelerde çalışan, ağ üzerinden yapılan bir API çağrısıysa ve ağ trafiği ile haberleşme maliyeti önemliyse bu refactoring yapılmamalıdır.

Soru 5 (ekrandan, 02:20:00): `printRecipt()` metodunda `thisAmount` geçici değişkeni kaldırılıp yerine `item.getCharge()` yazılırsa 1000 öğelik bir listede `getCharge()` kaç kez çağrılır? Bu değişiklik yapılmalı mıdır?

> **Çözüm (derste verildi):** Döngüde iki kez çağrıldığı için $2 \cdot 1000 = 2000$ kez çağrılır (önceden 1000). Sistemin asıl darboğazı büyük olasılıkla başka yerdedir; değişiklik yeni bir darboğaz oluşturmuyorsa yapılır. Kuşku varsa profiler ile ölçülür; başarım düşmüşse küçük adım geri alınır.

Soru 6 (ekrandan, 02:10:00): Preserve Whole Object düzenlemesinin iki zayıf yönü nedir ve bu düzenleme ne zaman kullanılmamalıdır?

Çözüm (derste verildi): 1) Parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bağımlılık oluşabilir. 2) Metot imzası değiştiği için geriye doğru uyumluluk kaybedilir ve refactoring tanımı esnetilir. Bu durumlardan biri bile çok sakıncalıysa düzenleme kullanılmaz; ikisi de boşa düşüyorsa çekinmeden yapılır.

Dersin Toparlaması

Bu derste önce Mediator kalıbıyla tasarım kalıpları tamamlandı. Font seçme penceresi örneğinde bileşenler arasındaki karesel büyüyen bağlantı sayısı, etkileşimi tek bir merkezde toplayan Mediator ile azaltıldı. Kalıbın bedeli, yeniden kullanılamayan ve büyüyen bir Mediator sınıfıdır; bu yüzden iyi tanımlanmış ama karmaşık etkileşimlerde kullanılır.

Ardından refactoring'e girildi: kodun ne yaptığını ve dışarıya açtığı kullanım biçimini değiştirmeden iç yapısını iyileştirmek. Refactoring'i tetikleyen işaretler kod kusurlarıdır ve her adım birim sınamasıyla doğrulanmalıdır. Bu yüzden JUnit 3 ve 4 tanıtıldı ve Fowler'ın video kiralama örneği için `TestMovie`, `TestCustomer` ve `TestAll` sınama sınıfları çalıştırıldı; bilerek yapılan bir hatanın sınamada nasıl yakalandığı gösterildi. Son bölümde Duplicated Code, Long Method ve Long Parameter List kusurları ile Extract Method, Pull Up Method, Form Template Method, Decompose Conditional, Introduce Parameter Object, Preserve Whole Object, Replace Temp with Query ve Replace Method with Method Object eylemleri işlendi. Hoca video kiralama örneğinde bütüncül refactoring'in sonraki derslerde adım adım yapılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.